

SSAFY

테스트 방법

유닛 테스트

공통



목 차

Unit Test

- UnitTest 실습
- UnitTest 주의사항

Unit Test

AI는 코드를 완벽하게 작성할까? (1/2)

- AI를 활용하면 프롬프트 몇 줄만으로 빠르게 잘 돌아가는 코드를 작성할 수 있음

<프롬프트>

html, javascript, css를 활용해서 내 포인트 잔액을 확인하고, 입력한 포인트만큼 차감하는 페이지를 만들어줘.

1. 만약 잔액이 사용할 금액보다 적으면 '잔액이 부족합니다.'라고 출력해줘.
2. 잔액이 충분하면 잔액에서 사용할 금액을 빼고 '남은 잔액은 00원입니다.'라고 출력해줘.

포인트 잔액 확인 및 사용

현재 잔액: 5,000원

5000

사용하기

남은 잔액은 5,000원입니다.

❖ 모델에 따라 다른 결과가 나올 수 있습니다.

AI는 코드를 완벽하게 작성할까? (2/2)

- 하지만 여러 번의 기능 추가 및 반복적인 수정을 하다보면, 기존의 잘 만들어진 기능이 망가지는 경우가 생김

<프롬프트>

방금 짠 코드에서 기능을 하나 추가해줘.

이제 포인트를 사용할 때마다 '수수료 500원'이 추가로 더 깎이게 코드를 수정해줘

포인트 잔액 확인 및 사용

현재 잔액: 0원

사용하기

수수료 500원 포함, 남은 잔액은 0원입니다.

❖ 모델에 따라 다른 결과가 나올 수 있습니다.

- 포인트 차감 시 수수료를 포함하지 않아서, 잔액 검사하는 기존 기능이 동작하지 않음

AI가 작성한 코드를 어떻게 검증할 수 있을까?

1. 수 많은 기능 추가 및 수정이 일어날 때마다 기존 기능이 잘 동작하는지 직접 테스트하기
 - 시간 자체가 부족할 뿐만 아니라, 테스트 항목을 놓칠 수 있음
2. 최종 개발이 다 끝나고 테스트하기
 - 코드가 엉켜 모든 기능이 꼬여버릴 수 있음
 - 언제부터 코드가 망가졌는지 파악하기 어려움

➤ **검증을 컴퓨터에게 맡기자!**

Unit Test(단위 테스트) (1/2)

- 컴퓨터에게 기능 단위로 검증 시키는 방법
- 모든 기능들이 정상적으로 동작하는지 편하게 테스트할 수 있음
- 테스트 코드가 동일한 입력을 넣었을 때, 동일한 출력값을 확인하는 것
- Sample Codes

```
// a와 b 값이 들어오면 더해주는 코드 (ai 가 작성한 코드 )  
function 더하기(a, b) {  
    return a + b;  
}
```

```
// 2와 3을 "더하기"에 넣으면 5가 잘 나오는지 확인하는 코드(단위 테스트 코드)  
test("더하기 함수는 2와 3을 넣으면 5를 뱉어야 한다", () => {  
    let x = 2;  
    let y = 3;  
    let 결과 = 더하기(x, y);  
    expect(결과).toBe(5);  
})
```

Unit Test(단위 테스트) (2/2)

- 기존 기능이 동일한 출력값을 내는지 확인하는 방식으로 동작
- 언제든지 빠르게 기존 기능이 올바르게 동작하는지 확인할 수 있음
- 많은 기능들에 대해서 각각의 테스트코드를 작성해야 하므로 개발 시간이 증가함
 - 개발 시간이 증가하지만, AI 활용으로 인해 개발 생산성이 증가했고,
 - 이제 단위 테스트 작성은 아주 쉬워졌음

Unit Test 실습

Unit Test 실습: 계산기 (1/5)

- **바이브코딩**을 통해 계산기를 구현하고, 계산기 기능에 대한 Unit Test를 만들어보자.
- 실습 내용
 - HTML, CSS, JavaScript 활용
 - 기존 계산기 기능에 대한 UnitTest 를 만들기
 - 여러 기능 추가 및 수정 시 Unit Test를 통해 기존 기능이 정상적으로 동작하는지 확인하기
- 주의사항
 - 사용하는 AI 모델에 따라 다른 결과가 나올 수 있음
 - 주어진 실습 내용 외에 다양한 시도를 하면서 학습 진행할 것

Unit Test 실습: 계산기 (2/5)

1. 바이브코딩으로 계산기 구현하기

<프롬프트>

html, javascript, css를 활용해서 두 개의 숫자를 입력받아 계산하는 심플한 계산기 페이지를 만들어줘.

<기능>

1. 첫 번째 숫자를 넣는 칸(input)과 두 번째 숫자를 넣는 칸(input), 그리고 [+] 버튼과 [-] 버튼을 만들어줘.
2. [+] 버튼을 누르면 두 숫자를 더해서 화면에 '더하기 결과는 OO입니다.'라고 출력해줘.
3. [-] 버튼을 누르면 첫 번째 숫자에서 두 번째 숫자를 빼서 화면에 '빼기 결과는 OO입니다.'라고 출력해줘.

1.cal_init.html

심플 계산기

+

-

더하기 결과는 8입니다.

Unit Test 실습: 계산기 (3/5)

2. 덧셈과 뺄셈 기능에 대한 단위 테스트 작성하기

<프롬프트>

방금 추가한 기능(덧셈, 뺄셈)이 제대로 작동하는지 검증하는 유닛 테스트 코드를 작성해줘.

[조건]

- 외부 라이브러리(Jest 등)나 어려운 개발 개념은 절대 쓰지 말아줘.
- 기존에 만든 계산기 코드 밑에 테스트 결과를 볼 수 있도록, 하나의 파일에 이어서 작성해줘
- 절대 기존에 만든 계산기 코드는 수정하지 마

2.test.html

유닛 테스트 결과 리포트

✅ [PASS] 덧셈 기능 검증 (5 + 3)

측정 결과: "더하기 결과는 8입니다." 정상 출력됨

✅ [PASS] 뺄셈 기능 검증 (10 - 4)

측정 결과: "빼기 결과는 6입니다." 정상 출력됨

Unit Test 실습: 계산기 (4/5)

3. 새로운 기능을 추가하고, 기존 기능(덧셈, 뺄셈)이 잘 동작하는지 확인하기

- 새로운 기능 추가에 대한 테스트 코드도 확인

<프롬프트>

아래 코드에 새로운 기능을 추가해줘.

<기능>

빼기 결과가 음수(<0)가 되면 결과를 무조건 0으로 고쳐줘.

3.updated_cal.html

심플 계산기

3

7

+

-

빼기 결과는 0입니다.

유닛 테스트 결과 리포트

- ✅ [PASS] 덧셈 기능 검증 (5 + 3)
측정 결과: "더하기 결과는 8입니다." 정상 출력됨
- ✅ [PASS] 일반 뺄셈 검증 (10 - 4)
측정 결과: "빼기 결과는 6입니다." 정상 출력됨
- ✅ [PASS] 뺄셈 음수 제한 검증 (3 - 7)
측정 결과: "빼기 결과는 0입니다." 정상 출력됨 (음수 방어 성공)

Unit Test 실습: 계산기 (5/5)

4. 일부로 기존 기능을 망가뜨려서, 테스트해보기

<프롬프트>

테스트 코드는 수정하지 말고, 기존 더하기 기능을 실행하면 최종 결과값에 +1 을 한 값을 출력하도록 코드를 수정해줘

4.failure_cal.html

심플 계산기

+

-

더하기 결과는 9입니다.

유닛 테스트 결과 리포트

❌ [FAIL] 덧셈 기능 검증 (5 + 3)

원인: 기대값은 "더하기 결과는 8입니다." 이
지만 실제로는 "더하기 결과는 9입니다." 가
출력됨

✅ [PASS] 일반 뺄셈 검증 (10 - 4)

측정 결과: "빼기 결과는 6입니다." 정상 출력
됨

✅ [PASS] 뺄셈 음수 제한 검증 (3 - 7)

측정 결과: "빼기 결과는 0입니다." 정상 출력
됨 (음수 방어 성공)

Unit Test 주의사항

Unit Test 주의사항

- 테스트 코드를 통과했다는 것이 버그가 없다는 것이 아님!
 - 우리가 짠 테스트코드는 작성한 기능에 대해서만 테스트를 해주는 것 뿐
 - 내가 놓친 테스트에 대해서는 여전히 에러가 발생할 수 있음
 - 테스트 코드를 너무 맹신하지 말 것
- 테스트 코드 작성에 많은 시간을 쏟지 말 것
 - 오류가 발생하면 안되는 핵심 로직에만 테스트 코드를 작성할 것



감사합니다

